LAPORAN PRAKTIKUM

STRUKTUR DATA

“DOUBLE LINKED LIST”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “SINGLE LINKED LIST” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya Single Linked List pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, krirtik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Manfaat Praktikum 1
BAB II **3**
2.1 Langkah Kerja Praktikum 3
2.2 Analisis Hasil dan Pembahasan 13
BAB III **16**
3.1 Kesimpulan 16
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi di bidang pemrograman membuat struktur data menjadi salah satu materi penting untuk dipelajari, khususnya dalam proses pengolahan data. Salah satu struktur data yang sering digunakan adalah Double Linked List. Struktur data ini memiliki kemampuan untuk menghubungkan setiap node ke node berikutnya maupun node sebelumnya, sehingga proses penelusuran dan pengelolaan data dapat dilakukan dengan lebih fleksibel.

Pada praktikum ini dilakukan implementasi beberapa operasi dasar pada Double Linked List menggunakan bahasa pemrograman Java, seperti penambahan node, penghapusan node, serta penelusuran data secara maju dan mundur. Dengan memahami konsep tersebut, mahasiswa dapat mengetahui bagaimana data disimpan dan dihubungkan dalam memori secara dinamis.

Melalui praktikum ini, diharapkan mahasiswa mampu memahami cara kerja Double Linked List serta dapat mengimplementasikan operasi-operasi dasar yang ada di dalamnya. Selain itu, praktikum ini juga membantu mahasiswa melatih logika pemrograman dan ketelitian dalam mengatur hubungan antar node agar struktur data dapat berjalan dengan baik.

1.2 Tujuan Praktikum

1. Memahami konsep dasar Doubly Linked List pada struktur data.
2. Mengetahui cara membuat node dan menghubungkan node pada Doubly Linked List.
3. Mengimplementasikan operasi penambahan node di awal, akhir, dan posisi tertentu.
4. Mengimplementasikan operasi penghapusan node pada Doubly Linked List.
5. Memahami proses penelusuran data secara maju dan mundur pada Doubly Linked List menggunakan Java.

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami penerapan struktur data Doubly Linked List secara langsung melalui pemrograman Java.

2. Mahasiswa dapat melatih kemampuan logika dan pemecahan masalah dalam mengelola data dinamis.
3. Membantu mahasiswa memahami cara kerja pointer atau hubungan antar node pada struktur data bertaut.
4. Menambah pengalaman dalam membuat program yang melibatkan operasi penambahan, penghapusan, dan penelusuran data.
5. Menjadi dasar pembelajaran untuk memahami struktur data lain yang lebih kompleks pada materi berikutnya.

BAB II
PEMBAHASAN

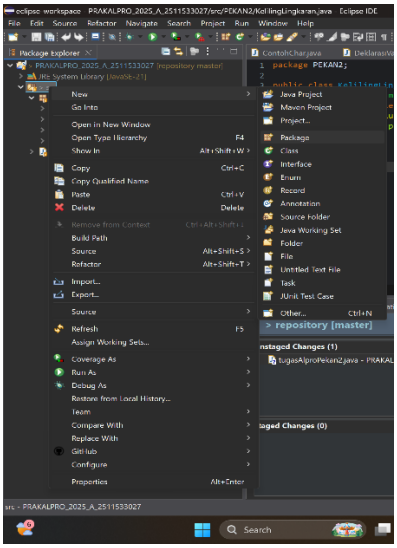
2.1 Langkah Kerja Praktikum

1. Persiapan Awal

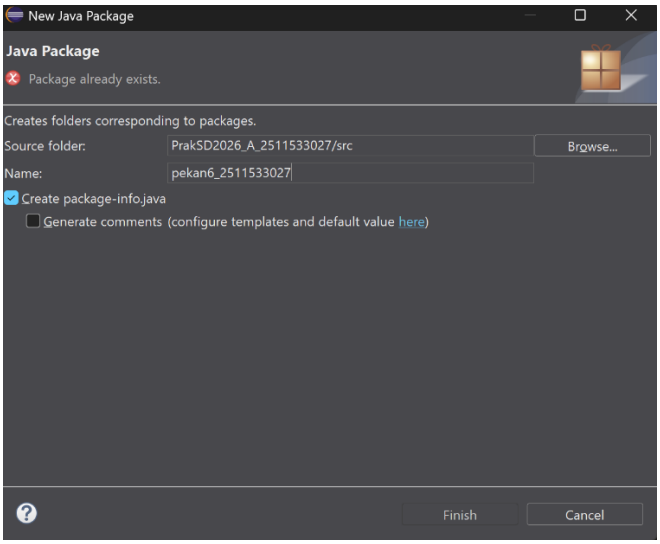
   a) Buka apl Eclipse IDE di laptop atau di komputer.

2. Membuat Package

   a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

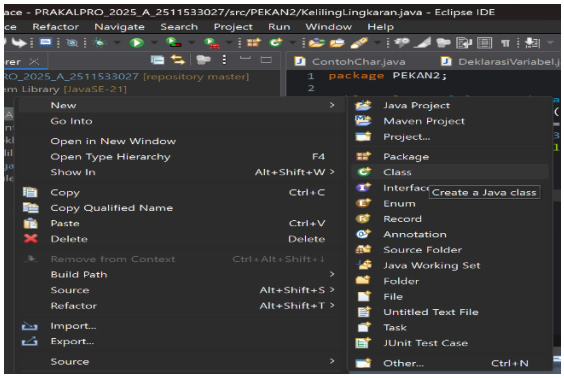


   b) Beri nama package **pekan6_NIM**.



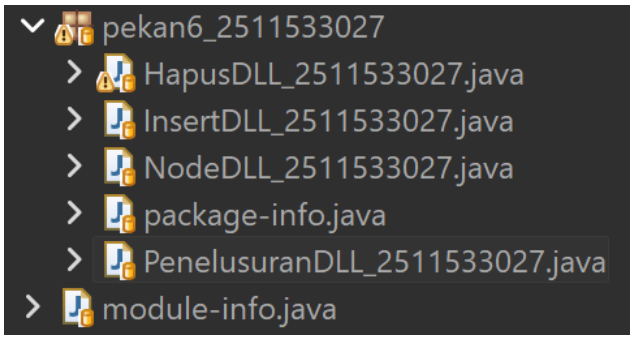
3. Membuat Class Baru untuk Setiap Percobaan

   a) Klik kanan pada package **pekan6_NIM**, lalu pilih **New**, terakhir klik **Class**.



   b) Buat beberapa kelas sesuai percobaan.

      - HapusDLL_2511533027.java
      - InsertDLL_2511533027.java
      - NodeDLL_2511533027.java
      - PenelusuranDLL_2511533027.java



4. Menulis Kode Program

   1. HapusDLL_2511533027.java

      - Program ini digunakan untuk memahami proses penghapusan node pada struktur data Double Linked List.
      - Program memiliki method delHead_3027() yang digunakan untuk menghapus node pada bagian awal linked list.
      - Method delLast_3027() digunakan untuk menghapus node terakhir pada linked list.
      - Method delPos_3027() digunakan untuk menghapus node pada posisi tertentu di dalam linked list.
      - Program juga memiliki method printList_3027() untuk menampilkan isi linked list setelah proses penghapusan data dilakukan.

- Pada method main, program membuat Double Linked List awal kemudian melakukan beberapa proses penghapusan node.
- Hasil akhir program berupa tampilan linked list sebelum dan sesudah penghapusan data dilakukan.

```
package pekan6_2511533027;

public class HapusDLL_2511533027 {
    // fungsi menghapus node awal
    public static NodeDLL_2511533027 delHead_3027(NodeDLL_2511533027
head_3027) {
        if (head_3027 == null) {
            return null;        }
        NodeDLL_2511533027 temp_3027 = head_3027;
        head_3027 = head_3027.next_3027;
        if (head_3027 != null) {
            head_3027.prev_3027 = null;             }
        return head_3027;
    }
    // fungsi menghapus di akhir
    public static NodeDLL_2511533027 delLast_3027(NodeDLL_2511533027
head_3027) {
        if (head_3027 == null) {
            return null;        }
        if (head_3027.next_3027 == null) {
            return null;        }
        NodeDLL_2511533027 curr_3027 = head_3027;
        while (curr_3027.next_3027 != null) {
            curr_3027 = curr_3027.next_3027;
        }
        // update pointer previous node
        if (curr_3027.prev_3027 != null) {
            curr_3027.prev_3027.next_3027 = null;         }
        return head_3027;
    }
    // fungsi menghapus node posisi tertentu
    public static NodeDLL_2511533027 delPos_3027(NodeDLL_2511533027
head_3027, int pos_3027) {
        // jika DLL kosong
```

```
        if (head_3027 == null) {
                return head_3027;          }
        NodeDLL_2511533027 curr_3027 = head_3027;
        // telusuri sampai ke node yang akan di hapus
        for (int i = 1; curr_3027 != null && i < pos_3027; ++i) {
                curr_3027 = curr_3027.next_3027;        }
        // jika posisi tidak ditemukan
        if (curr_3027 == null) {
                return head_3027;          }
        // update pointer
        if (curr_3027.prev_3027 != null) {
                curr_3027.prev_3027.next_3027 = curr_3027.next_3027;
    }
        if (curr_3027.next_3027 != null) {
                curr_3027.next_3027.prev_3027 = curr_3027.prev_3027;
    }
        // jika yang di hapus head
        if (head_3027 == curr_3027) {
                head_3027 = curr_3027.next_3027;        }
        return head_3027;
    }
    // fungsi mencetak DLL
    public static void printList_3027(NodeDLL_2511533027 head_3027) {
        NodeDLL_2511533027 curr_3027 = head_3027;
        while (curr_3027 != null) {
                System.out.print(curr_3027.data_3027 + " <-> ");
                curr_3027 = curr_3027.next_3027;
        }
         System.out.println();
    }
    public static void main(String[] args) {
        // buat sebuah DLL
        NodeDLL_2511533027 head_3027 = new NodeDLL_2511533027(1);
        head_3027.next_3027 = new NodeDLL_2511533027(2);
        head_3027.next_3027.prev_3027 = head_3027;
        head_3027.next_3027.next_3027 = new NodeDLL_2511533027(3);
        head_3027.next_3027.next_3027.prev_3027 = head_3027.next_3027;
        head_3027.next_3027.next_3027.next_3027          =          new
NodeDLL_2511533027(4);
```

```
        head_3027.next_3027.next_3027.next_3027.prev_3027 =
head_3027.next_3027.next_3027;
        head_3027.next_3027.next_3027.next_3027.next_3027 = new
NodeDLL_2511533027(5);
        head_3027.next_3027.next_3027.next_3027.next_3027.prev_3027 =
head_3027.next_3027.next_3027.next_3027;

        System.out.print("DLL Awal: ");
        printList_3027(head_3027);

        System.out.print("Setelah head dihapus: ");
        head_3027 = delHead_3027(head_3027);
        printList_3027(head_3027);

        System.out.print("Setelah node terakhir dihapus: ");
        head_3027 = delLast_3027(head_3027);
        printList_3027(head_3027);

        System.out.print("Menghapus node ke 2: ");
        head_3027 = delPos_3027(head_3027, 2);

        printList_3027(head_3027);
    }

}
```

2. InsertDLL_2511533027.java
   - Program ini digunakan untuk memahami proses penambahan node pada struktur data Double Linked List.
   - Program memiliki method insertBegin_3027() yang digunakan untuk menambahkan node di awal linked list.
   - Method insertEnd_3027() digunakan untuk menambahkan node di akhir linked list dengan menelusuri node terakhir terlebih dahulu.
   - Method insertAtPosition_3027() digunakan untuk menambahkan node pada posisi tertentu di dalam linked list.
   - Program juga memiliki method printList_3027() untuk menampilkan isi linked list setelah proses penambahan data dilakukan.

- Pada method main, program membuat Double Linked List awal kemudian melakukan beberapa proses penambahan node.
- Hasil akhir program berupa tampilan linked list sebelum dan sesudah penambahan data dilakukan.

```
package pekan6_2511533027;

public class InsertDLL_2511533027 {
    // menambahkan node di awal DLL
    static     NodeDLL_2511533027     insertBegin_3027(NodeDLL_2511533027
head_3027, int data_3027) {
          // buat node baru
          NodeDLL_2511533027         new_node_3027         =         new
NodeDLL_2511533027(data_3027);
          // jadikan pointer nextnya head
          new_node_3027.next_3027 = head_3027;
          // jadikan pointer prev head ke new_node
          if (head_3027 != null) {
                head_3027.prev_3027 = new_node_3027;
          }
          return new_node_3027;
    }

    // fungsi menambahkan node di akhir
    public  static  NodeDLL_2511533027  insertEnd_3027(NodeDLL_2511533027
head_3027, int newData_3027) {
          // buat node baru
          NodeDLL_2511533027         newNode_3027         =         new
NodeDLL_2511533027(newData_3027);
          // jika dll nu;; jadikan head
          if (head_3027 == null) {
                head_3027 = newNode_3027;
          }
          else {
                NodeDLL_2511533027 curr_3027 = head_3027;
                while (curr_3027.next_3027 != null) {
                      curr_3027 = curr_3027.next_3027;
                }
                curr_3027.next_3027 = newNode_3027;
```

```
                newNode_3027.prev_3027 = curr_3027;
        }
        return head_3027;
    }

    // fungsi menambahkan node di posisi tertentu
    public static NodeDLL_2511533027 insertAtPosition_3027(NodeDLL_2511533027 head_3027, int pos_3027, int new_data_3027) {
        // buat node baru
        NodeDLL_2511533027 new_node_3027 = new NodeDLL_2511533027(new_data_3027);
        if (pos_3027 == 1) {
                new_node_3027.next_3027 = head_3027;
                if (head_3027 != null) {
                        head_3027.prev_3027 = new_node_3027;    }
                head_3027 = new_node_3027;
                return head_3027;     }
        NodeDLL_2511533027 curr_3027 = head_3027;
    for (int i = 1; i < pos_3027 - 1 && curr_3027 != null; ++i) {
                curr_3027 = curr_3027.next_3027;    }
        if (curr_3027 == null) {
                System.out.println("Posisi tidak ada");
                return head_3027;     }
        new_node_3027.prev_3027 = curr_3027;
        new_node_3027.next_3027 = curr_3027.next_3027;
        curr_3027.next_3027 = new_node_3027;
        if (new_node_3027.next_3027 != null) {
                new_node_3027.next_3027.prev_3027 = new_node_3027;   }
        return head_3027;
    }
    public static void printList_3027(NodeDLL_2511533027 head_3027) {
        NodeDLL_2511533027 curr_3027 = head_3027;
        while (curr_3027 != null) {
                System.out.print(curr_3027.data_3027 + " <-> ");
                curr_3027 = curr_3027.next_3027;
        }
        System.out.println();
    }
```

```java
    public static void main(String[] args) {
        // membuat dll 2 <-> 3 <-> 5
        NodeDLL_2511533027 head_3027 = new NodeDLL_2511533027(2);
        head_3027.next_3027 = new NodeDLL_2511533027(3);
        head_3027.next_3027.prev_3027 = head_3027;
        head_3027.next_3027.next_3027 = new NodeDLL_2511533027(5);
        head_3027.next_3027.next_3027.prev_3027 = head_3027.next_3027;
        // cetak DLL awal
        System.out.print("DLL Awal: ");
        printList_3027(head_3027);
        // tambah 1 di awal
        head_3027 = insertBegin_3027(head_3027, 1);
        System.out.print("Simpul 1 ditambah di awal: ");
        printList_3027(head_3027);
        // tambah 6 di akhir
        System.out.print("Simpul 6 ditambah di akhir: ");
        int data = 6;
        head_3027 = insertEnd_3027(head_3027, data);
        printList_3027(head_3027);
        // menambah 4 node di posisi 4
        System.out.print("Tambah node 4 di posisi 4: ");
        int data2_3027 = 4;
        int pos_3027 = 4;
        head_3027    =    insertAtPosition_3027(head_3027,    pos_3027,
data2_3027);
        printList_3027(head_3027);
    }

}
```

3. NodeDLL_2511533027.java
    - Program ini digunakan untuk mendefinisikan node pada struktur data Double Linked List.
    - Pada kelas ini terdapat atribut data_3027 yang digunakan untuk menyimpan nilai data pada node.
    - Atribut next_3027 digunakan sebagai pointer yang menunjuk ke node berikutnya.
    - Atribut prev_3027 digunakan sebagai pointer yang menunjuk ke node sebelumnya.

- Program memiliki constructor NodeDLL_2511533027() yang berfungsi untuk menginisialisasi data node serta mengatur pointer next dan prev menjadi null.
- Kelas ini menjadi dasar utama dalam pembuatan struktur Double Linked List pada program lainnya.

```java
package pekan6_2511533027;

public class NodeDLL_2511533027 {
    // mendefinisikan kelas Node
    int data_3027; // data
    NodeDLL_2511533027 next_3027; // Pointer ke next node
    NodeDLL_2511533027 prev_3027; // Pointer ke previous node

    // konstruktor
    public NodeDLL_2511533027(int data_3027) {
        this.data_3027 = data_3027;
        this.next_3027 = null;
        this.prev_3027 = null;
    }

}
```

4. PencarianDLL_2511533027.java
    - Program ini digunakan untuk memahami proses penelusuran pada struktur data Double Linked List.
    - Program memiliki method forwardTraversal_3027() yang digunakan untuk melakukan penelusuran data dari depan ke belakang.
    - Method backwardTraversal_3027() digunakan untuk melakukan penelusuran data dari belakang ke depan.
    - Penelusuran maju dilakukan menggunakan pointer next_3027, sedangkan penelusuran mundur menggunakan pointer prev_3027.
    - Pada method main, program membuat beberapa node lalu menghubungkannya menjadi Double Linked List.
    - Program kemudian menampilkan hasil penelusuran maju dan mundur dari linked list yang telah dibuat.

- Hasil akhir program berupa tampilan isi linked list sesuai arah penelusuran yang dilakukan.

```
package pekan6_2511533027;

public class PenelusuranDLL_2511533027 {
    // fungsi penelusuran maju
    static void forwardTraversal_3027(NodeDLL_2511533027 head_3027) {
            // memulai penelusuran dari head
            NodeDLL_2511533027 curr_3027 = head_3027;
            // lanjutkan sampai akhir
            while (curr_3027 != null) {
                    // print data
                    System.out.print(curr_3027.data_3027 + " <-> ");
                    // pindah ke node berikutnya
                    curr_3027 = curr_3027.next_3027;
            }
            // print spasi
             System.out.println();
    }
    // fungsi penelusuran mundur
    static void backwardTraversal_3027(NodeDLL_2511533027 tail_3027) {
            // mulai dari akhir
            NodeDLL_2511533027 curr_3027 = tail_3027;
            // lanjut sampai head
            while (curr_3027 != null) {
                    // cetak data
                     System.out.print(curr_3027.data_3027 + " <-> ");
                    // pindah ke node sebelumnya
                     curr_3027 = curr_3027.prev_3027;
            }
            // cetak spasi
             System.out.println();
    }
    public static void main(String[] args) {
            // cetak DLL
            NodeDLL_2511533027 head_3027 = new NodeDLL_2511533027(1);
            NodeDLL_2511533027 second_3027 = new NodeDLL_2511533027(2);
            NodeDLL_2511533027 third_3027 = new NodeDLL_2511533027(3);
```

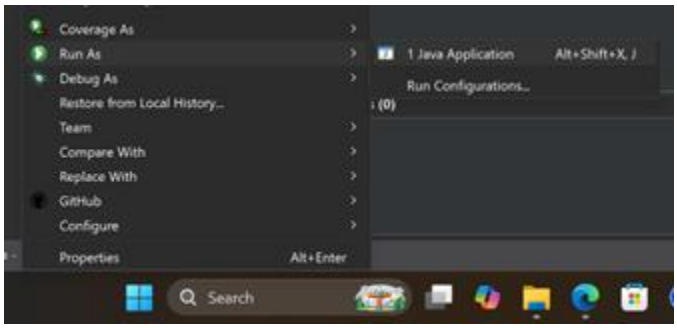
```
        head_3027.next_3027 = second_3027;
        second_3027.prev_3027 = head_3027;
        second_3027.next_3027 = third_3027;
        third_3027.prev_3027 = second_3027;

        System.out.println("Penelusuran maju:");
        forwardTraversal_3027(head_3027);

        System.out.println("Penelusuran mundur:");
        backwardTraversal_3027(third_3027);
    }

}
```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. HapusDLL_2511533027.java

   a) Hasil: Program berhasil melakukan proses penghapusan node pada Double Linked List. Penghapusan dapat dilakukan pada bagian awal linked list, bagian akhir, maupun pada posisi tertentu sesuai input yang diberikan. Setelah proses penghapusan dilakukan, isi linked list berubah sesuai node yang telah dihapus dan hasilnya dapat ditampilkan dengan benar menggunakan method printList_3027().

   b) Pembahasan: Pada program ini diterapkan beberapa operasi dasar penghapusan data pada Double Linked List. Method delHead_3027() bekerja dengan memindahkan pointer head ke node berikutnya sehingga node pertama tidak lagi terhubung dengan linked list. Method delLast_3027() digunakan untuk menghapus node terakhir dengan cara menelusuri node hingga mencapai node terakhir lalu memutus hubungan dengan node sebelumnya. Selain itu, method delPos_3027() digunakan untuk menghapus node berdasarkan posisi tertentu dengan mengatur kembali pointer next dan prev pada node di sekitarnya. Dari

hasil percobaan dapat dilihat bahwa setiap proses penghapusan memengaruhi hubungan antar node pada Double Linked List sehingga pointer harus diperbarui dengan benar agar linked list tetap berjalan dengan baik.

2. InsertDLL_2511533027.java
   a) Hasil: Program berhasil melakukan proses penambahan node pada Double Linked List. Penambahan data dapat dilakukan di bagian awal linked list, di bagian akhir, maupun pada posisi tertentu. Setelah proses penambahan dilakukan, isi linked list berubah sesuai node yang ditambahkan dan hasilnya dapat ditampilkan dengan benar menggunakan method printList_3027().
   b) Pembahasan: Pada program ini diterapkan beberapa operasi dasar penambahan node pada Double Linked List. Method insertBegin_3027() digunakan untuk menambahkan node di awal linked list dengan mengubah pointer head ke node baru. Method insertEnd_3027() digunakan untuk menambahkan node di akhir linked list dengan menelusuri node terakhir terlebih dahulu. Selain itu, method insertAtPosition_3027() digunakan untuk menambahkan node pada posisi tertentu dengan mengatur pointer next dan prev antar node agar hubungan linked list tetap terhubung dengan baik. Dari hasil percobaan dapat dilihat bahwa setiap proses penambahan memengaruhi hubungan antar node pada Double Linked List.
3. NodeDLL_2511533027.java
   a) Hasil: Program berhasil membuat node pada struktur data Double Linked List. Setiap node dapat menyimpan data serta memiliki pointer yang menghubungkan node ke bagian berikutnya dan sebelumnya. Constructor pada program juga berhasil menginisialisasi nilai data dan pointer next maupun prev dengan benar.
   b) Pembahasan: Pada program ini dibuat kelas NodeDLL_2511533027 yang digunakan sebagai dasar pembentukan Double Linked List. Setiap node memiliki atribut data_3027 untuk menyimpan nilai data, next_3027 untuk menunjuk node berikutnya, dan prev_3027 untuk menunjuk node sebelumnya. Constructor digunakan untuk memberikan nilai awal pada node saat objek dibuat. Dengan adanya kelas node ini, proses penelusuran, penambahan, dan penghapusan data pada Double Linked List dapat dilakukan dengan lebih mudah karena hubungan antar node sudah terbentuk melalui pointer next dan prev.
4. PencarianDLL_2511533027.java
   a) Hasil: Program berhasil melakukan penelusuran data pada Double Linked List secara maju maupun mundur. Penelusuran maju menampilkan data dari node

pertama hingga node terakhir, sedangkan penelusuran mundur menampilkan data dari node terakhir menuju node pertama. Hasil penelusuran berhasil ditampilkan sesuai urutan data yang terdapat pada linked list.

b) Pembahasan: Pada program ini diterapkan proses traversal atau penelusuran pada Double Linked List. Method forwardTraversal_3027() digunakan untuk melakukan penelusuran maju dengan memanfaatkan pointer next_3027 hingga mencapai node terakhir. Sedangkan method backwardTraversal_3027() digunakan untuk melakukan penelusuran mundur menggunakan pointer prev_3027. Dari hasil percobaan terlihat bahwa Double Linked List memiliki kelebihan dibanding Single Linked List karena dapat ditelusuri dari dua arah. Hubungan antar node yang menggunakan pointer next dan prev membuat proses traversal menjadi lebih fleksibel.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Doubly Linked List memungkinkan setiap node terhubung ke node berikutnya dan node sebelumnya. Hal tersebut membuat proses penelusuran data dapat dilakukan dari dua arah, yaitu maju dan mundur.

Melalui praktikum ini juga dapat dipahami cara melakukan beberapa operasi dasar pada Doubly Linked List, seperti penambahan node, penghapusan node, dan penelusuran data. Setiap operasi dilakukan dengan mengatur pointer next dan prev agar hubungan antar node tetap tersusun dengan benar.

Selain memahami konsep Doubly Linked List, praktikum ini juga membantu melatih kemampuan logika pemrograman dan ketelitian dalam membuat program menggunakan bahasa Java. Dengan adanya praktikum ini, mahasiswa dapat lebih memahami cara kerja struktur data dinamis serta penerapannya dalam pemrograman.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Double Linked List*. Padang: PPT Perkuliahan, 2026.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson, 2012.

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“DOUBLE LINKED LIST”

disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadirat Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Double Linked List menggunakan studi kasus Playlist Lagu sederhana. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data Website seperti menambah, menampilkan, dan menghapus data.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR ..... **i**
DAFTAR ISI ..... **ii**
BAB I **1**
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 2
1.4 Instruksi Tugas 2
BAB II **4**
2.1 Kelas ADT (Lagu_NIM.java) 4
2.2 Kelas Driver (Musik_NIM.java) 5
BAB III **9**
3.1 Kesimpulan 9
DAFTAR PUSTAKA .....

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi di bidang pemrograman membuat pengelolaan data menjadi hal yang penting dalam pembuatan suatu program. Salah satu cara untuk mengelola data secara dinamis adalah dengan menggunakan struktur data linked list. Double Linked List merupakan pengembangan dari linked list yang memungkinkan setiap node terhubung ke node berikutnya dan node sebelumnya, sehingga proses penelusuran data dapat dilakukan dari dua arah.

Pada tugas praktikum ini, Double Linked List diterapkan dalam pembuatan program playlist musik sederhana. Setiap lagu disimpan sebagai node yang memiliki informasi judul lagu, penyanyi, serta pointer next dan prev. Dengan menggunakan struktur data ini, program dapat melakukan proses penambahan lagu, penghapusan lagu, penelusuran playlist, dan pencarian lagu dengan lebih teratur.

Melalui tugas ini, mahasiswa dapat memahami penerapan Double Linked List dalam kehidupan sehari-hari, khususnya pada sistem playlist musik. Selain itu, tugas ini juga membantu melatih kemampuan logika pemrograman dan pemahaman terhadap hubungan antar node pada struktur data bertaut.

1.2 Tujuan

1. Memahami konsep dasar Doubly Linked List dalam bahasa pemrograman Java.
2. Mengimplementasikan struktur data Doubly Linked List pada program playlist musik sederhana.
3. Melatih kemampuan membuat operasi penambahan, penghapusan, penelusuran, dan pencarian data pada linked list.
4. Memahami penggunaan pointer next dan prev dalam menghubungkan node pada Doubly Linked List.
5. Melatih kemampuan logika pemrograman dalam menyelesaikan permasalahan pengelolaan data.

1.3 Manfaat

1. Mahasiswa dapat memahami penerapan struktur data Doubly Linked List secara langsung melalui program playlist musik.
2. Membantu melatih kemampuan berpikir logis dan sistematis dalam membuat program.
3. Menambah pemahaman tentang hubungan antar node menggunakan pointer next dan prev.
4. Memberikan pengalaman dalam membuat program interaktif menggunakan bahasa Java.
5. Menjadi dasar pembelajaran untuk memahami struktur data dan algoritma yang lebih kompleks pada materi berikutnya.

1.4 Instruksi Tugas

1. Aturan Penamaan
   a. Nama Kelas: pakai NIM lengkap. Contoh: Lagu_2411531005 dan Musik_2411531005
   b. Nama variabel & method: pakai 4 digit terakhir NIM. Contoh: judul_1005, tambahLagu_1005()
   c. Pointer: next_1005 dan prev_1005 juga wajib pakai akhiran NIM
2. Struktur yang Dibuat
   a. Kelas ADT (Lagu_NIM.java)
      - Atribut yang harus ada:
        • judul (String)
        • penyanyi (String)
        • next (pointer ke Lagu)
        • prev (pointer ke Lagu)
      - Method:
        • constructor
        • getter
        • setter
   b. Kelas Driver (Musik_NIM.java)
      - Buat 5 operasi utama saja (nama method diakhiri 4 digit NIM):
        1. tambahLagu_xxxx() : menambah lagu baru di AKHIR playlist
        2. hapusLaguAwal_xxxx() : menghapus lagu pertama (head)

3. tampilMaju_xxxx() : menampilkan semua lagu dari awal ke akhir
4. tampilMundur_xxxx() : menampilkan semua lagu dari akhir ke awal (wajib DLL)
5. cariLagu_xxxx(judul) : mencari lagu berdasarkan judul (tidak case-sensitive)

3. Contoh Tampilan Menu

```
=== Playlist Musik NIM: 2411531005 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar

Pilihan: 1
Judul: Rosalía - Mio Cristo Piange Diamanti
Penyanyi: Rosalía
Lagu berhasil ditambahkan!
```

BAB II
PEMBAHASAN

2.1 Kelas ADT (Lagu_NIM.java)

Kelas Lagu_2511533027 digunakan untuk merepresentasikan data lagu pada program playlist musik menggunakan struktur data Doubly Linked List. Pada kelas ini terdapat atribut judul_3027 dan penyanyi_3027 yang digunakan untuk menyimpan informasi lagu. Selain itu, terdapat pointer next_3027 dan prev_3027 yang berfungsi untuk menghubungkan node ke lagu berikutnya dan lagu sebelumnya. Kelas ini juga memiliki constructor untuk menginisialisasi data lagu saat objek dibuat, serta getter dan setter yang digunakan untuk mengambil dan mengubah data lagu. Kelas Lagu_2511533027 menjadi dasar utama dalam pembentukan playlist musik pada program.

```java
package pekan6_2511533027;

public class Lagu_2511533027 {
    // atribut lagu
    String judul_3027;
    String penyanyi_3027;

    // pointer next dan prev
    Lagu_2511533027 next_3027;
    Lagu_2511533027 prev_3027;

    // constructor
    public Lagu_2511533027(String judul_3027, String penyanyi_3027) {
        this.judul_3027 = judul_3027;
        this.penyanyi_3027 = penyanyi_3027;
        this.next_3027 = null;
        this.prev_3027 = null;
    }

    // getter judul
    public String getJudul_3027() {
        return judul_3027;
    }

    // setter judul
    public void setJudul_3027(String judul_3027) {
        this.judul_3027 = judul_3027;
    }

    // getter penyanyi
    public String getPenyanyi_3027() {
        return penyanyi_3027;
    }

    // setter penyanyi
    public void setPenyanyi_3027(String penyanyi_3027) {
        this.penyanyi_3027 = penyanyi_3027;
    }

}
```

2.2 Kelas Driver (Musik_NIM.java)

Kelas Musik_2511533027 digunakan untuk mengelola playlist lagu menggunakan struktur data Doubly Linked List. Pada kelas ini terdapat pointer head_3027 dan tail_3027 yang digunakan untuk menunjuk node pertama dan node terakhir pada playlist. Program memiliki beberapa method utama, seperti tambahLagu_3027() untuk menambahkan lagu di akhir playlist, hapusLaguAwal_3027() untuk menghapus lagu pertama, tampilMaju_3027() untuk menampilkan playlist dari awal ke akhir, tampilMundur_3027() untuk menampilkan playlist dari akhir ke awal, serta cariLagu_3027() untuk mencari lagu berdasarkan judul. Pada method main, program menampilkan menu interaktif agar pengguna dapat mengelola playlist lagu sesuai pilihan yang diinginkan. Dengan adanya kelas ini, proses pengelolaan data lagu menjadi lebih teratur dan mudah dipahami.

```java
package pekan6_2511533027;

import java.util.Scanner;

public class Musik_2511533027 {
    // head dan tail DLL
    Lagu_2511533027 head_3027 = null;
    Lagu_2511533027 tail_3027 = null;

    // method tambah lagu di akhir
    public void tambahLagu_3027(String judul_3027, String penyanyi_3027) {
        Lagu_2511533027 laguBaru_3027 = new Lagu_2511533027(judul_3027, penyanyi_3027);

        // jika playlist kosong
        if (head_3027 == null) {
            head_3027 = laguBaru_3027;
            tail_3027 = laguBaru_3027;
        }


     // tambah di akhir
        else {
            tail_3027.next_3027 = laguBaru_3027;
            laguBaru_3027.prev_3027 = tail_3027;
            tail_3027 = laguBaru_3027;
        }

        System.out.println("\nLagu berhasil ditambahkan!");
    }
     // method hapus lagu awal
        public void hapusLaguAwal_3027() {
            // jika playlist kosong
            if (head_3027 == null) {
                System.out.println("Playlist masih kosong!\n");
                return;
            }

            System.out.println("Lagu " + head_3027.judul_3027 + " berhasil dihapus!\n");

            // pindahkan head
            head_3027 = head_3027.next_3027;
```

```java
        if (head_3027 != null) {
            head_3027.prev_3027 = null;
        }

        else {
            tail_3027 = null;
        }
    }

// tampil playlist maju
    public void tampilMaju_3027() {
        // cek playlist kosong
        if (head_3027 == null) {
            System.out.println("Playlist kosong!\n");
            return;
        }

        Lagu_2511533027 curr_3027 = head_3027;

        System.out.println("=== Playlist Maju ===");

        while (curr_3027 != null) {
            System.out.println(curr_3027.judul_3027 + " - " + curr_3027.penyanyi_3027);
            curr_3027 = curr_3027.next_3027;
        }

        System.out.println();
    }

// tampil playlist mundur
    public void tampilMundur_3027() {
        // cek playlist kosong
        if (tail_3027 == null) {
            System.out.println("Playlist kosong!\n");
            return;
        }

        Lagu_2511533027 curr_3027 = tail_3027;

        System.out.println("=== Playlist Mundur ===");
```

```java
        while (curr_3027 != null) {
            System.out.println(curr_3027.judul_3027 + " - " + curr_3027.penyanyi_3027);
            curr_3027 = curr_3027.prev_3027;
        }

        System.out.println();
    }

// cari lagu berdasarkan judul
    public void cariLagu_3027(String judulCari_3027) {
        // cek playlist kosong
        if (head_3027 == null) {
            System.out.println("Playlist kosong!\n");
            return;
        }

        Lagu_2511533027 curr_3027 = head_3027;
        boolean ditemukan_3027 = false;

        while (curr_3027 != null) {
            if (curr_3027.judul_3027.equalsIgnoreCase(judulCari_3027)) {
                System.out.println("Lagu ditemukan!");
                System.out.println("Judul    : " + curr_3027.judul_3027);
                System.out.println("Penyanyi : " + curr_3027.penyanyi_3027 + "\n");
                ditemukan_3027 = true;
                break;
            }

            curr_3027 = curr_3027.next_3027;
        }

        if (!ditemukan_3027) {
            System.out.println("Lagu tidak ditemukan!\n");
        }
    }

    // method utama
    public static void main(String[] args) {
        Scanner input_3027 = new Scanner(System.in);
        Musik_2511533027 playlist_3027 = new Musik_2511533027();
```

```java
        int pilihan_3027;

        do {
            System.out.println("=== Playlist Musik NIM: 2511533027 ===");
            System.out.println("1. Tambah Lagu");
            System.out.println("2. Hapus Lagu Pertama");
            System.out.println("3. Lihat Playlist (Maju)");
            System.out.println("4. Lihat Playlist (Mundur)");
            System.out.println("5. Cari Lagu");
            System.out.println("6. Keluar");
            System.out.print("Pilihan: ");

            pilihan_3027 = input_3027.nextInt();
            input_3027.nextLine();

         switch (pilihan_3027) {
            case 1:
                System.out.print("Judul Lagu : ");
                String judul_3027 = input_3027.nextLine();

                System.out.print("Penyanyi   : ");
                String penyanyi_3027 = input_3027.nextLine();

                playlist_3027.tambahLagu_3027(judul_3027, penyanyi_3027);
                break;

            case 2:
                playlist_3027.hapusLaguAwal_3027();
                break;

            case 3:
                playlist_3027.tampilMaju_3027();
                break;

            case 4:
                playlist_3027.tampilMundur_3027();
                break;
```

```java
             case 5:
                 System.out.print("Masukkan judul lagu yang dicari: ");
                 String cari_3027 = input_3027.nextLine();
                 playlist_3027.cariLagu_3027(cari_3027);
                 break;

             case 6:
                 System.out.println("Program selesai 🎖");
                 break;

             default:
                 System.out.println("Pilihan tidak tersedia!\n");
         }

        } while (pilihan_3027 != 6);

         input_3027.close();
    }

}
```

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan tugas praktikum yang telah dikerjakan, program playlist musik sederhana berhasil dibuat menggunakan struktur data Doubly Linked List. Program mampu menjalankan beberapa fitur utama seperti menambahkan lagu, menghapus lagu pertama, menampilkan playlist secara maju dan mundur, serta mencari lagu berdasarkan judul.

Penggunaan pointer next dan prev pada setiap node membuat data lagu dapat diakses dari dua arah sehingga proses penelusuran playlist menjadi lebih mudah dan fleksibel. Selain itu, program juga dapat berjalan dengan baik saat playlist kosong maupun ketika data lagu ditambahkan dan dihapus.

Melalui tugas ini, dapat dipahami bahwa Doubly Linked List sangat cocok digunakan dalam pengelolaan data yang membutuhkan hubungan dua arah antar node, seperti playlist musik. Tugas ini juga membantu melatih kemampuan dalam menyusun logika program serta memahami penerapan struktur data dalam bahasa Java.

DAFTAR PUSTAKA

[1] Wahyudi, *Materi Struktur Data: Double Linked List*. Padang: PPT Perkuliahan, 2026.

[2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson, 2012.